Terceira Lista

Nome: Lucas do Nascimento Macedo
Matricula: 11811BCC044

Código 1

```
import numpy as np

p=0.6 # probabilidade de sair cara
nsim = 10 # num de experimentos
nhead = 0 # num de caras obtidas
saida = [] # armazena as saidas (cara:1, coroa:0)

for i in range(0, nsim):
    if(np.random.uniform() <p): # se menor que p, cara
        nhead = nhead+1 # incrementa o contador de caras
        saida.append(1)
    else:
        saida.append(0)

print("Saida: ", saida)
print("Frenquencia de caras: ", nhead/nsim)
```

A saída para esse código será:

```
lucas@lucas-Aspire-E5-574:~/Dropbox/UFU/6º Periodo/Modelagem e Simulação/Terceira Lista de Exercicios/Códigos$ python3 video1
.py
Saida:  [1, 1, 1, 1, 1, 0, 0, 1, 1, 1]
Frenquencia de caras:  0.8
```

Código 2

```
import numpy as np
import matplotlib.pyplot as plt

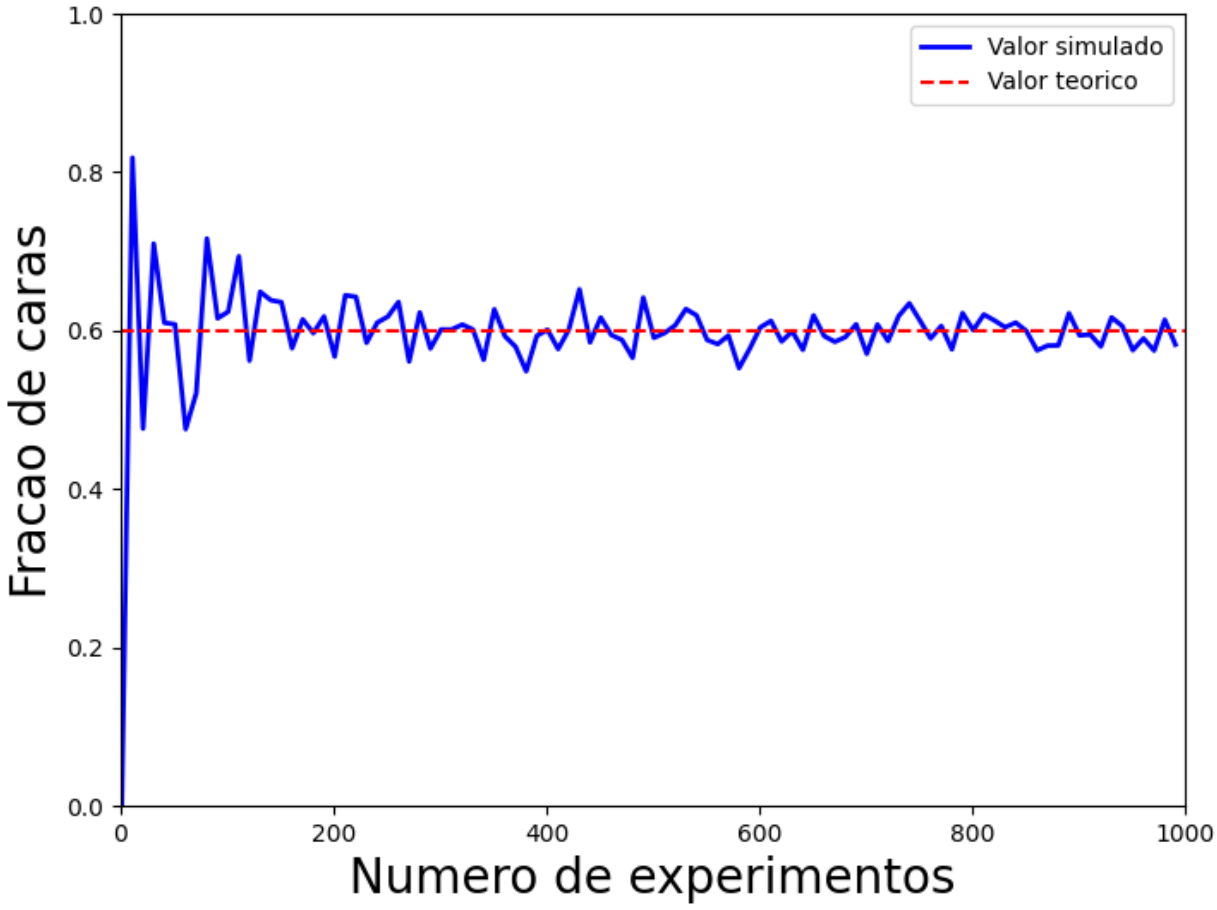
p=0.6 # chance de sair cara
vp = [] # lista de armazena a frequencia de ocorrencias
vsim = [] #armazena o numero de simulações
Nmax = 1000 # numero máimo de simulações

for nsim in np.arange(1, Nmax, 10): #simula de 1 até Nmax
    nhead = 0 # numero de caras
    for i in range(1, nsim):  #o numero de simulações aumenta
        if np.random.uniform()<p:
            nhead = nhead+1
    vp.append(nhead/nsim)
    vsim.append(nsim)

#Mostra os resultados em uma figura
```

```
plt.figure(figsize=(8,6))
plt.plot(vsim, vp, linestyle='-', color="blue", linewidth=2, label='Valor simulado')
plt.axhline(y=p, color='r', linestyle='--', label = 'Valor teorico')
plt.ylabel("Fracao de caras", fontsize=20)
plt.xlabel("Numero de experimentos", fontsize=20)
plt.xlim([0.0, Nmax])
plt.ylim([0.0, 1.0])
plt.legend()
plt.show()
```

A saída para esse código será:

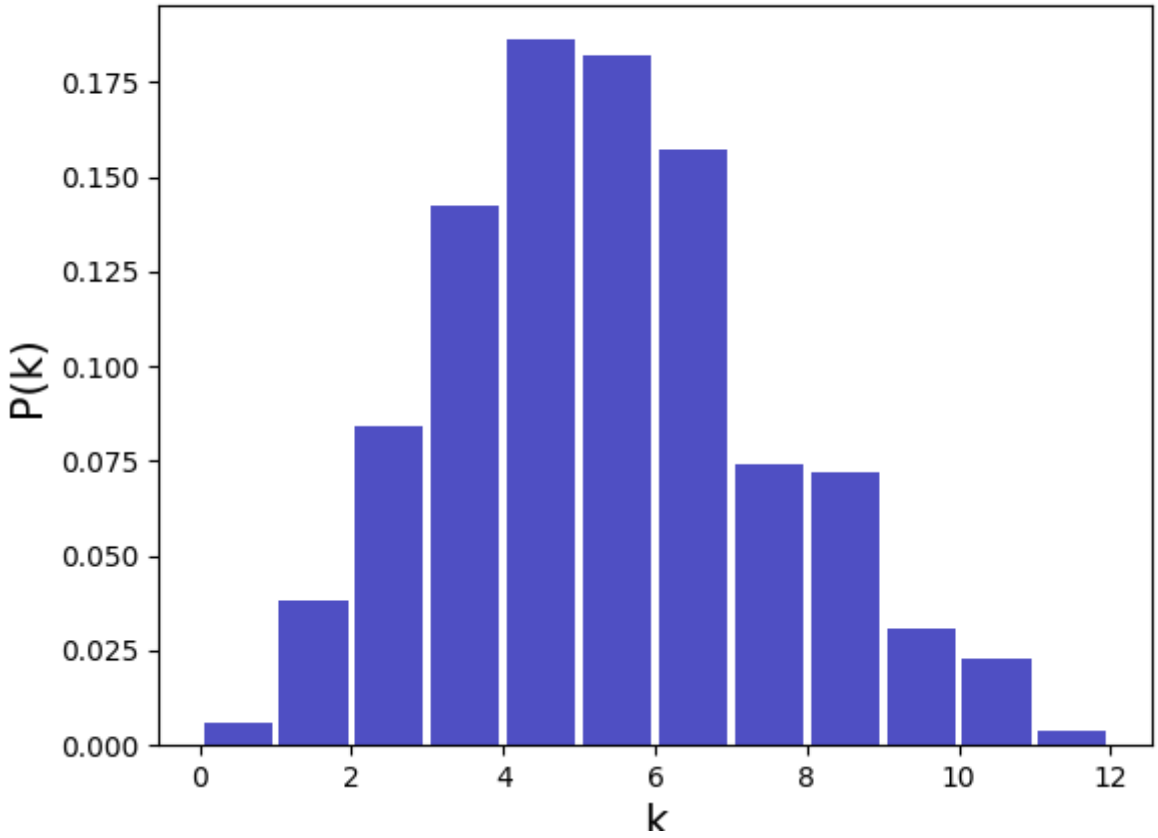


Código 3

```
import numpy as np
import matplotlib.pyplot as plt

lbd = 5 # Taxa
n=1000 # numero de pontos entraidos de uma distribuicao de Poisson
p = np.random.poisson(lbd, n)
k = np.arange(0, np.max(p))
count, bins, ignored = plt.hist(p, bins=k, density=True, color= "#0504aa", alpha=0.7,
rwidth=0.9)
plt.xlabel('k', fontsize= 15)
plt.ylabel('P(k)', fontsize= 15)
plt.show()
```

A saída para esse código será:



Código 4

```
from random import seed
from matplotlib import pyplot as plt
import numpy as np
from scipy.stats import binom
from scipy.special import factorial

seed(100) # semente do gerador de numeros aleatórios

vp = [0.5, .02, 0.05] # probabilidade de sucesso
n = 100 # numero de experimentos
for p in vp:
    Pk = np.zeros(n)
    vk = np.arange(0, n)
    ns = 1000 # numero de simulações

    for j in range(0, ns): #faca para ns simulacoes
        S = 0 # numero de sucessos
        for i in range(0, n): # faça para n experimentos
            r = np.random.uniform()
            if(r<=p): # se o sucesso
                S = S+1
        Pk[S] = Pk[S]+1

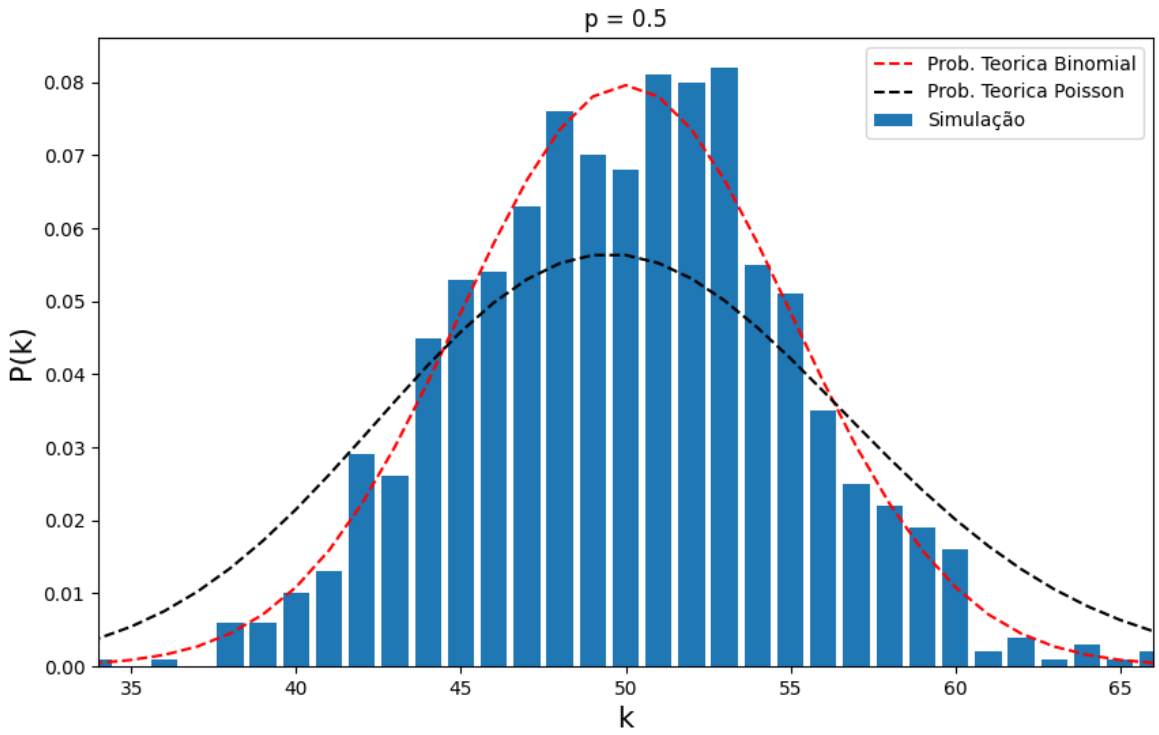
    Pk= Pk/sum(Pk) #normaliza a distribuição de probabilidade
    #plt.plot(vk, Pk, 'ro')
    plt.figure(figsize=(10, 6))
    plt.xlim(np.min(vk[Pk>0]), np.max(vk[Pk>0]))
    plt.bar(vk, Pk, label='Simulação')

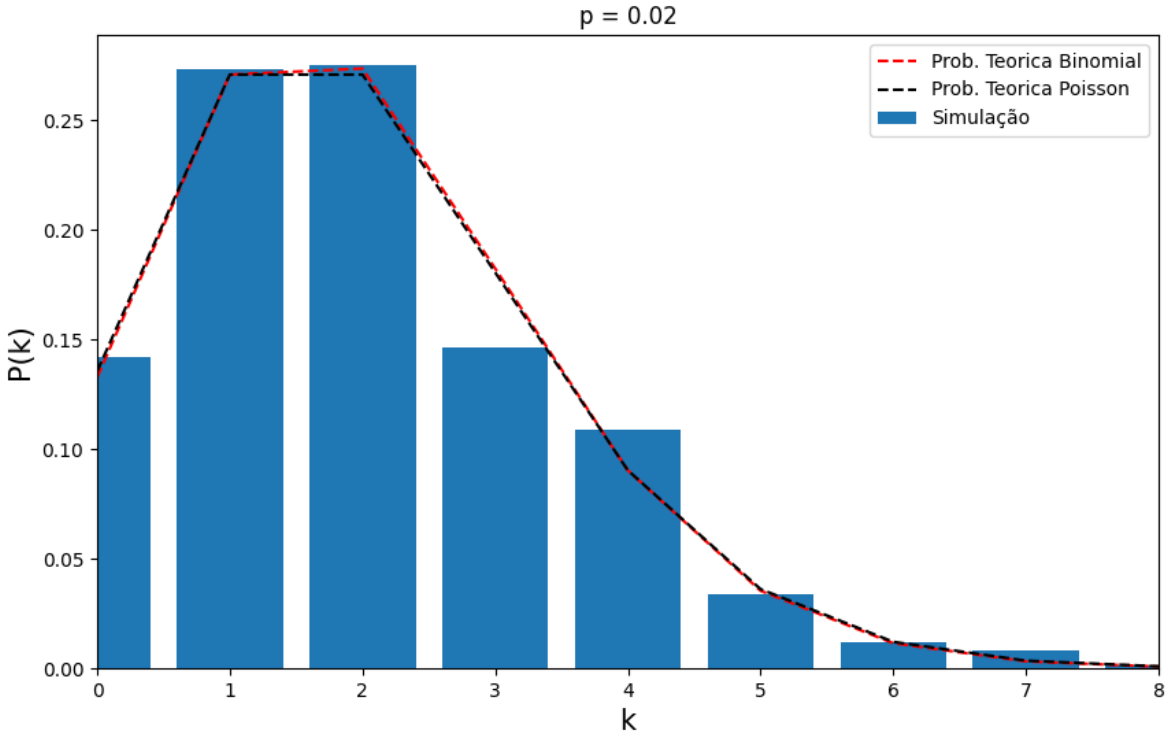
    #curva teorica
    Pkt = np.zeros(n+1) # valores teoricos da probabilidade
```

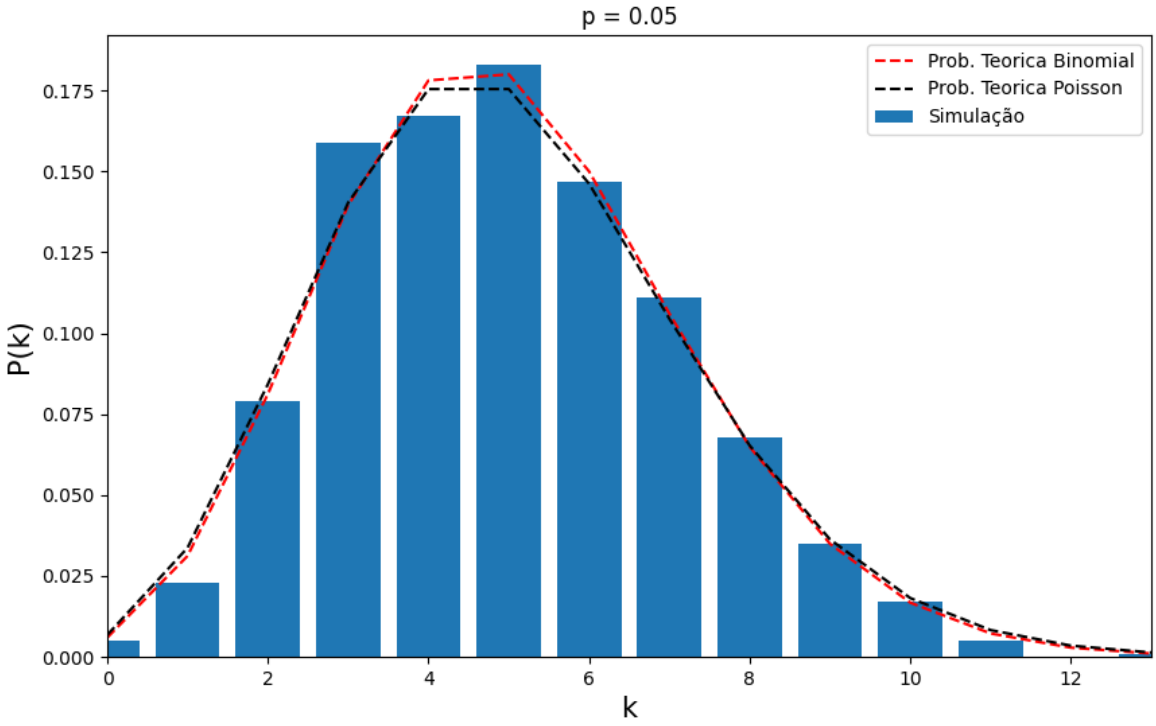
```
vkt = np.arange(0, n+1) # variacao em k
for k in range(0, n+1): # varia de 0 até n
    Pkt[k] = (factorial(n)/(factorial(n-k)*factorial(k)))*(p**k)*(1-p)**(n-k)
plt.plot(vkt, Pkt, 'r--', label='Prob. Teorica Binomial')

#curva teorica
Po = np.zeros(n+1) # valores teoricos da probabilidade
vo = np.arange(0, n+1) # variacao em k
lbd = n*p
for k in range(0, n+1): # varia de 0 até n
    Po[k] = (np.exp(-lbd)*lbd**k)/factorial(k)
plt.plot(vo, Po, 'k--', label= 'Prob. Teorica Poisson')
plt.title('p = '+str(p))
plt.xlabel('k', fontsize= 15)
plt.ylabel('P(k)', fontsize= 15)
plt.legend()
plt.show()
```

A saída para esse código será:




p = 0.02
Prob. Teorica Binomial
Prob. Teorica Poisson
Simulação
P(k)
k
0.00
0.05
0.10
0.15
0.20
0.25
0
1
2
3
4
5
6
7
8


p = 0.05
Prob. Teorica Binomial
Prob. Teorica Poisson
Simulação
P(k)
k
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0
2
4
6
8
10
12